

UNIT 3: STACKS, QUEUES, RECURSION

3.1 Stacks

A **Stack** is a linear data structure in which insertion and deletion of elements take place only at one end, called the **TOP**.

A stack follows the **LIFO (Last In, First Out)** principle.

This means that the element inserted last is removed first.

Real-Life Example

A stack of plates is a good example of a stack data structure.

If plates are placed in this order:

Plate 1 → Plate 2 → Plate 3

The last plate placed, **Plate 3**, will be removed first.

Basic Structure

TOP → [30]
 [20]
 [10]

Here, **30** is the top element and will be removed first.

Applications of Stack

Stacks are used in:

- Function calls and recursion.
- Expression evaluation.
- Expression conversion.
- Undo and redo operations.
- Browser back button.
- Parentheses matching.
- Backtracking algorithms.
- Memory management.

3.2 Array Representation of Stacks

A stack can be implemented using an **array**.

An array stores the elements of the stack, while a variable called **TOP** keeps track of the position of the topmost element.

Consider a stack of size 5:

Index Element

0 10

1 20

2 30

3 40

4 50

Here:

TOP = 4

The element **50** is the top element.

Empty Stack

When the stack contains no elements:

TOP = -1

Full Stack

For an array of size **n**, the stack is full when:

TOP = n - 1

Example

Initially:

TOP = -1

After inserting 10:

TOP = 0

After inserting 20:

TOP = 1

After inserting 30:

TOP = 2

Stack:

TOP → 30
 20
 10

3.3 Operations on Stacks

The main operations performed on a stack are:

1. Push
 2. Pop
 3. Peek
 4. IsEmpty
 5. IsFull
-

1. Push Operation

Push means adding a new element to the top of the stack.

Example:

Initial stack:

TOP → 30
20
10

Push 40:

TOP → 40
30
20
10

Algorithm

1. Check whether the stack is full.
2. If the stack is full, report **Stack Overflow**.
3. Otherwise, increase TOP by 1.
4. Insert the new element at TOP.

Time Complexity

O(1)

2. Pop Operation

Pop means removing the topmost element from the stack.

Example:

Initial stack:

TOP → 40
30
20
10

After Pop:

TOP → 30
20
10

The element **40** is removed.

Algorithm

1. Check whether the stack is empty.
2. If the stack is empty, report **Stack Underflow**.
3. Otherwise, access the element at TOP.
4. Decrease TOP by 1.

Time Complexity

O(1)

3. Peek Operation

Peek returns the top element of the stack without removing it.

Example:

TOP → 40
30
20

Peek returns:

40

The stack remains unchanged.

4. IsEmpty Operation

This operation checks whether the stack contains any elements.

If:

TOP = -1

then the stack is empty.

5. IsFull Operation

This operation checks whether the stack has reached its maximum capacity.

For an array of size **n**:

TOP = n - 1

means the stack is full.

Stack Overflow

Stack Overflow occurs when we try to **push** an element into a full stack.

Stack Underflow

Stack Underflow occurs when we try to **pop** an element from an empty stack.

3.4 Arithmetic Expression

An **arithmetic expression** consists of operands and operators.

Example

A + B × C

Here:

- **A, B, C** are operands.
- **+, ×** are operators.

There are three common forms of arithmetic expressions:

1. Infix Expression
 2. Prefix Expression
 3. Postfix Expression
-

1. Infix Expression

In an infix expression, the operator is placed **between the operands**.

Example

A + B

Another example:

(A + B) × C

This is the normal form used in mathematics.

2. Prefix Expression

In a prefix expression, the operator is placed **before the operands**.

It is also called **Polish Notation**.

Example

Infix:

A + B

Prefix:

+ A B

Another example:

Infix:

(A + B) × C

Prefix:

× + A B C

3. Postfix Expression

In a postfix expression, the operator is placed **after the operands**.

It is also called **Reverse Polish Notation (RPN)**.

Example

Infix:

A + B

Postfix:

A B +

Another example:

Infix:

(A + B) × C

Postfix:

A B + C ×

Use of Stack in Arithmetic Expressions

Stacks are widely used for:

- Converting infix expressions to postfix.
- Converting infix expressions to prefix.
- Evaluating postfix expressions.
- Evaluating prefix expressions.
- Handling parentheses and operator precedence.

Operator Precedence

Operators generally follow this order:

1. **Parentheses ()**
2. **Exponentiation ^**
3. **Multiplication, Division, Modulus *, / , %**
4. **Addition and Subtraction +, -**

For example:

$$A + B \times C$$

First, $B \times C$ is performed because multiplication has higher precedence than addition.

3.5 Queues

A **Queue** is a linear data structure in which insertion takes place at one end called the **REAR**, and deletion takes place at the other end called the **FRONT**.

A queue follows the **FIFO (First In, First Out)** principle.

This means that the element inserted first is removed first.

Real-Life Example

A queue of people waiting at a ticket counter is an example of a queue.

The first person who joins the line is served first.

Structure

FRONT → [10] [20] [30] [40] ← **REAR**

Here:

- **10** is the first element and will be removed first.
- **40** is the last element and was inserted most recently.

Applications of Queues

Queues are used in:

- Printer scheduling.
- CPU scheduling.
- Keyboard input buffering.
- Network data transmission.
- Customer service systems.
- Breadth-First Search (BFS).
- Operating system processes.

3.6 Queue Operations

The main operations performed on a queue are:

1. Enqueue
2. Dequeue
3. Peek
4. IsEmpty
5. IsFull

1. Enqueue Operation

Enqueue means adding an element to the **REAR** of the queue.

Example:

Initial queue:

FRONT → [10] [20] [30] ← **REAR**

Enqueue 40:

FRONT → [10] [20] [30] [40] ← **REAR**

The new element is added at the rear.

Algorithm

1. Check whether the queue is full.
2. If full, report **Queue Overflow**.
3. Otherwise, increase REAR.
4. Insert the new element at REAR.

Time Complexity

O(1)

2. Dequeue Operation

Dequeue means removing an element from the **FRONT** of the queue.

Example:

Initial queue:

FRONT → [10] [20] [30] [40] ← **REAR**

After Dequeue:

FRONT → [20] [30] [40] ← REAR

The element **10** is removed first because the queue follows FIFO.

Algorithm

1. Check whether the queue is empty.
2. If empty, report **Queue Underflow**.
3. Remove the element at FRONT.
4. Move FRONT to the next position.

Time Complexity

O(1)

3. Peek Operation

Peek returns the element at the front without removing it.

Example:

FRONT → [10] [20] [30] ← REAR

Peek returns:

10

The queue remains unchanged.

4. IsEmpty Operation

This operation checks whether the queue contains any elements.

If the queue has no elements, it is called an **empty queue**.

5. IsFull Operation

This operation checks whether all available positions in the queue are occupied.

If no space is available for a new element, the queue is full.

Queue Overflow

Occurs when an element is inserted into a full queue.

Queue Underflow

Occurs when an element is removed from an empty queue.

3.7 Priority Queue

A **Priority Queue** is a special type of queue in which each element is assigned a priority. The element with the **highest priority** is processed before elements with lower priority. Unlike a normal queue, a priority queue does not always follow the strict FIFO principle.

Example

Suppose a hospital emergency department has the following patients:

Patient	Priority
---------	----------

Patient A	Low
-----------	-----

Patient B	High
-----------	------

Patient C	Medium
-----------	--------

The service order will be:

Patient B → Patient C → Patient A

The patient with the highest priority is served first.

Types of Priority Queue

1. Ascending Priority Queue

The element with the **smallest priority value** is removed first.

Example:

1 → 2 → 3 → 4

Here, **1** has the highest priority.

2. Descending Priority Queue

The element with the **largest priority value** is removed first.

Example:

4 → 3 → 2 → 1

Here, **4** has the highest priority.

Operations on Priority Queue

The main operations are:

1. **Insertion** – Adds an element with a priority.
2. **Deletion** – Removes the highest-priority element.
3. **Peek** – Returns the highest-priority element without removing it.

Applications of Priority Queue

Priority queues are used in:

- CPU scheduling.
- Dijkstra's shortest path algorithm.
- Prim's Minimum Spanning Tree algorithm.
- Network routing.
- Printer scheduling.
- Emergency service systems.
- Event-driven simulations.

Difference Between Stack and Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Full Form	Last In, First Out	First In, First Out
Insertion	At TOP	At REAR
Deletion	From TOP	From FRONT
Main Operations	Push, Pop	Enqueue, Dequeue
Example	Stack of plates	Queue at ticket counter
Applications	Recursion, undo, expression evaluation Scheduling, buffering, BFS	

Difference Between Normal Queue and Priority Queue

Feature	Normal Queue	Priority Queue
Ordering	FIFO	Based on priority
First Removal	Oldest element	Highest-priority element
Priority	Not considered	Considered
Example	Ticket queue	Emergency patient queue
Main Use	General sequential processing Priority-based processing	

Conclusion

A **Stack** is a linear data structure that follows the **LIFO** principle and performs operations such as push and pop from the top. A **Queue** follows the **FIFO** principle, where insertion occurs at the rear and deletion occurs from the front. **Priority Queues** process elements according to their priority

rather than simply following FIFO. Stacks are also important in handling arithmetic expressions, function calls, and recursion, while queues are widely used in scheduling, buffering, and computer networking.